

Java & OOP Programs

Complete Reference Guide — v2.0

Basics + OOP Concepts + Design Patterns + Java 8 + Multithreading

90+

Unique Programs

7

Categories

Senior

Level Focus

1 Numbers

1. Odd or Even Number

```
import java.util.Scanner;
public class OddEven {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int number = sc.nextInt();
        System.out.println(number % 2 == 0 ? number+" is even." : number+" is odd.");
    }
}
```

■ Output / Example:

Input: 4 → Output: 4 is even.

Input: 7 → Output: 7 is odd.

2. Prime Number Check

```
public class PrimeCheck {
    public static void main(String[] args) {
        int num = 11; boolean isPrime = true;
        for (int i = 2; i <= Math.sqrt(num); i++)
            if (num % i == 0) { isPrime = false; break; }
        System.out.println(num + (isPrime ? " is prime." : " is not prime.));
    }
}
```

■ Output / Example:

Input: 11 → is prime. | Input: 9 → is not prime.

3. Fibonacci Series

```
public class Fibonacci {
    public static void main(String[] args) {
        int n=10, a=0, b=1;
        System.out.print("Fibonacci: "+a+", "+b);
        for (int i=2; i<n; i++) {
            int c = a+b; System.out.print(", "+c); a=b; b=c;
        }
    }
}
```

■ Output / Example:

Output: Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

4. Fibonacci – Recursive

```
public class FibRecursive {
    static int printFibo(int n, int a, int b) {
        if (n == 0) return a;
        System.out.print(a + " ");
        return printFibo(n-1, b, a+b);
    }
    public static void main(String[] args) { printFibo(10, 0, 1); }
}
```

■ Output / Example:

Output: 0 1 1 2 3 5 8 13 21 34

■ 5. Factorial (Loop)

```
public class Factorial {
    public static void main(String[] args) {
        int num=5, fact=1;
        for (int i=1; i<=num; i++) fact*=i;
        System.out.println("Factorial: "+fact);
    }
}
```

■ Output / Example:

Input: 5 → Output: Factorial: 120

■ 6. Factorial – Recursive

```
public class FactRecursive {
    static int factorial(int n) { return n==0 ? 1 : n*factorial(n-1); }
    public static void main(String[] args) { System.out.println(factorial(5)); }
}
```

■ Output / Example:

Input: 5 → Output: 120

■ 7. Reverse a Number

```
public class ReverseNumber {
    public static void main(String[] args) {
        int num=12345, rev=0;
        while (num!=0) { rev=rev*10+num%10; num/=10; }
        System.out.println("Reversed: "+rev);
    }
}
```

■ Output / Example:

Input: 12345 → Output: Reversed: 54321

■ 8. Armstrong Number Check

```
public class Armstrong {
    public static void main(String[] args) {
        int num=153, sum=0, temp=num;
        while (temp!=0) { int d=temp%10; sum+=Math.pow(d,3); temp/=10; }
        System.out.println(num==sum ? "Armstrong":"Not Armstrong");
    }
}
```

■ Output / Example:

Input: 153 → Armstrong | Input: 123 → Not Armstrong

■ 9. Palindrome Number

```
public class PalindromeNum {
    static boolean isPalin(int n) {
        int orig=n, rev=0;
        while (n>0) { rev=rev*10+n%10; n/=10; }
        return orig==rev;
    }
    public static void main(String[] args) {
        System.out.println(isPalin(12321) ? "Palindrome":"Not Palindrome");
    }
}
```

■ Output / Example:

Input: 12321 → Palindrome

■ 10. Sum of Digits

```
public class SumDigits {  
    public static void main(String[] args) {  
        int num=12345, sum=0;  
        while (num!=0) { sum+=num%10; num/=10; }  
        System.out.println("Sum: "+sum);  
    }  
}
```

■ Output / Example:

Input: 12345 → Output: Sum: 15

■ 11. Count Digits

```
public class CountDigits {  
    public static void main(String[] args) {  
        int num=12345;  
        System.out.println("Digits: "+String.valueOf(num).length());  
    }  
}
```

■ Output / Example:

Input: 12345 → Output: Digits: 5

■ 12. Swap Two Numbers (No 3rd Variable)

```
public class Swap {  
    public static void main(String[] args) {  
        int a=5, b=10;  
        a=a+b; b=a-b; a=a-b;  
        System.out.println("a="+a+", b="+b);  
    }  
}
```

■ Output / Example:

Output: a=10, b=5

■ 13. GCD of Two Numbers

```
public class GCD {  
    public static void main(String[] args) {  
        int a=60, b=48;  
        while (b!=0) { int t=b; b=a%b; a=t; }  
        System.out.println("GCD: "+a);  
    }  
}
```

■ Output / Example:

Input: 60,48 → Output: GCD: 12

■ 14. Prime Numbers in Range

```
public class PrimeRange {  
    public static void main(String[] args) {  
        for (int n=10; n<=50; n++) {  
            boolean p=true;  
            for (int i=2; i<=Math.sqrt(n); i++) if(n%i==0){p=false;break;}  
            if (p && n>1) System.out.print(n+" ");  
        }  
    }  
}
```

■ Output / Example:

Output: 11 13 17 19 23 29 31 37 41 43 47

■ 15. Decimal to Binary

```
public class DecToBin {
    public static void main(String[] args) {
        System.out.println("Binary: "+Integer.toBinaryString(10));
    }
}
```

■ Output / Example:

Input: 10 → Output: Binary: 1010

■ 16. Perfect Number

```
public class PerfectNum {
    public static void main(String[] args) {
        int num=28, sum=0;
        for (int i=1; i<=num/2; i++) if(num%i==0) sum+=i;
        System.out.println(num==(sum) ? "Perfect":"Not Perfect");
    }
}
```

■ Output / Example:

Input: 28 → Output: Perfect

■ 17. Leap Year Check

```
public class LeapYear {
    public static void main(String[] args) {
        int y=2024;
        boolean leap=(y%4==0&& y%100!=0)|| (y%400==0);
        System.out.println(y+(leap?" is":" is not")+ " a leap year.");
    }
}
```

■ Output / Example:

Input: 2024 → is a leap year.

■ 18. Sum of First N Natural Numbers

```
public class SumNatural {
    public static void main(String[] args) {
        int n=10, sum=n*(n+1)/2;
        System.out.println("Sum: "+sum);
    }
}
```

■ Output / Example:

Input: 10 → Output: Sum: 55

■ 19. Simple Calculator

```
import java.util.Scanner;
public class Calculator {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        double a=sc.nextDouble(), b=sc.nextDouble();
        char op=sc.next().charAt(0);
        double r = op=='+'?a+b : op=='-'?a-b : op=='*'?a*b : a/b;
        System.out.println("Result: "+r);
    }
}
```

■ Output / Example:

Input: 10 5 + → Output: Result: 15.0

■ 20. Pascal's Triangle

```
public class PascalTriangle {
    public static void main(String[] args) {
        int rows=5;
        for (int i=0; i<rows; i++) {
            int n=1;
            System.out.format("%"+(rows-i)*2+"s", "");
            for (int j=0; j<=i; j++) {
                System.out.format("%4d", n); n=n*(i-j)/(j+1);
            }
            System.out.println();
        }
    }
}
```

■ Output / Example:

```
Output:
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

■ 21. Missing Number in Array

```
public class MissingNum {
    public static void main(String[] args) {
        int[] arr={1,2,4,5,6};
        int n=arr.length+1, total=n*(n+1)/2;
        for (int x:arr) total-=x;
        System.out.println("Missing: "+total);
    }
}
```

■ Output / Example:

```
Input: {1,2,4,5,6} → Output: Missing: 3
```

■ 2 Strings

■ 22. Reverse a String

```
public class ReverseStr {
    public static void main(String[] args) {
        String s="Automation";
        System.out.println(new StringBuilder(s).reverse());
    }
}
```

■ Output / Example:

```
Output: noitamotua
```

■ 23. Reverse Each Word

```
public class ReverseWords {
    public static void main(String[] args) {
        String[] words = "Java is good".split(" ");
        StringBuilder sb = new StringBuilder();
        for (String w : words) sb.append(new StringBuilder(w).reverse()).append(" ");
        System.out.println(sb.toString().trim());
    }
}
```

■ Output / Example:

```
Input: Java is good → Output: avaJ si doog
```

■ 24. Palindrome String

```
public class PalinStr {
    public static void main(String[] args) {
        String s="madam";
        System.out.println(s.equals(new StringBuilder(s).reverse().toString()))
        ? "Palindrome":"Not Palindrome");
    }
}
```

■ Output / Example:

Input: madam → Output: Palindrome

■ 25. Anagram Check

```
import java.util.Arrays;
public class Anagram {
    public static void main(String[] args) {
        char[] a="listen".toCharArray(), b="silent".toCharArray();
        Arrays.sort(a); Arrays.sort(b);
        System.out.println(Arrays.equals(a,b) ? "Anagram":"Not Anagram");
    }
}
```

■ Output / Example:

Input: listen, silent → Output: Anagram

■ 26. Count Vowels and Consonants

```
public class VowelCons {
    public static void main(String[] args) {
        String s="Hello World"; int v=0,c=0;
        for (char ch : s.toLowerCase().toCharArray()) {
            if (ch>='a'&&ch<='z') { if("aeiou".indexOf(ch)>=0) v++; else c++; }
        }
        System.out.println("Vowels:"+v+" Consonants:"+c);
    }
}
```

■ Output / Example:

Output: Vowels:3 Consonants:7

■ 27. Find Duplicate Characters

```
import java.util.HashMap;
public class DupChars {
    public static void main(String[] args) {
        String s="Learn Java"; HashMap<Character,Integer> m=new HashMap<>();
        for (char c:s.toCharArray()) m.put(c,m.getOrDefault(c,0)+1);
        m.forEach((k,v)->{ if(v>1) System.out.println(k+": "+v); });
    }
}
```

■ Output / Example:

Output: a:4 r:2 (characters with count > 1)

■ 28. Count Word Occurrences

```
import java.util.HashMap;
public class WordOccur {
    public static void main(String[] args) {
        String s="Test Auto Java Auto"; HashMap<String,Integer> m=new HashMap<>();
        for (String w:s.split(" ")) m.put(w,m.getOrDefault(w,0)+1);
        System.out.println(m);
    }
}
```

■ Output / Example:

Output: {Java=1, Auto=2, Test=1}

■ 29. Count Words

```
public class CountWords {
    public static void main(String[] args) {
        String s="Welcome to Java World"; int c=1;
        for (int i=0;i<s.length()-1;i++)
            if(s.charAt(i)=='&&s.charAt(i+1)!=' ') c++;
        System.out.println("Words: "+c);
    }
}
```

■ Output / Example:

Output: Words: 4

■ 30. All Permutations of a String

```
public class Permutations {
    static void permute(String s, String pre) {
        if (s.isEmpty()) { System.out.println(pre); return; }
        for (int i=0;i<s.length();i++)
            permute(s.substring(0,i)+s.substring(i+1), pre+s.charAt(i));
    }
    public static void main(String[] args) { permute("abc",""); }
}
```

■ Output / Example:

Output: abc, acb, bac, bca, cab, cba

■ 31. Print Unique Characters

```
public class UniqueChars {
    public static void main(String[] args) {
        String s="Java Automation"; boolean[] seen=new boolean[128];
        for (char c:s.toCharArray()) if(!seen[c]){seen[c]=true;System.out.print(c+" ");}
    }
}
```

■ Output / Example:

Output: J a v A u t o m i n

■ 32. Remove Spaces

```
public class RemoveSpaces {
    public static void main(String[] args) {
        System.out.println("Hello World".replaceAll("\\s+", ""));
    }
}
```

■ Output / Example:

Output: HelloWorld

■ 33. Character Compression

```
public class CharCompress {
    public static void main(String[] args) {
        String s="aabbcccd"; String r=new StringBuilder(); int c=1;
        for (int i=0;i<s.length();i++) {
            if(i+1<s.length()&&s.charAt(i)==s.charAt(i+1)) c++;
            else { r.append(s.charAt(i)).append(c); c=1; }
        }
        System.out.println(r);
    }
}
```

■ Output / Example:

Input: aabbcccd → Output: a2b2c3d2

■ 34. Separate Upper and Lowercase

```
public class SeparateCase {
    public static void main(String[] args) {
        String s="aBACbcEded"; String lo=new StringBuilder(), up=new StringBuilder();
        for (char c:s.toCharArray()) { if(Character.isLowerCase(c)) lo.append(c); else up.append(c); }
        System.out.println("Lower:"+lo+" Upper:"+up);
    }
}
```

■ Output / Example:

Output: Lower:abced Upper:ABCED

■ 35. Separate Alpha and Numeric

```
public class SeparateAlphaNum {
    public static void main(String[] args) {
        String s="Subbu123raj"; String a=new StringBuilder(), n=new StringBuilder();
        for (char c:s.toCharArray()) { if(Character.isLetter(c)) a.append(c); else if(Character.isDigit(c)) n.append(c); }
        System.out.println("Alpha:"+a+" Numeric:"+n);
    }
}
```

■ Output / Example:

Input: Subbu123raj → Alpha:Subburaj Numeric:123

■ 36. Longest Substring Without Repeating

```
import java.util.HashSet;
public class LongestSubstr {
    static int longest(String s) {
        HashSet<Character> set=new HashSet<>(); int max=0,i=0,j=0;
        while (j<s.length()) {
            if(!set.contains(s.charAt(j))) { set.add(s.charAt(j)); max=Math.max(max,j-i); }
            else set.remove(s.charAt(i++));
        }
        return max;
    }
    public static void main(String[] args) { System.out.println(longest("abcabcbb")); }
}
```

■ Output / Example:

Input: abcabcbb → Output: 3

■ 37. Remove Special Characters

```
public class RemoveSpecial {
    public static void main(String[] args) {
        String s="Hello, World! @2024";
        System.out.println(s.replaceAll("[^a-zA-Z0-9\\s]", ""));
    }
}
```

■ Output / Example:

Output: Hello World 2024

■ 38. First Non-Repeated Character

```
import java.util.*;
public class FirstNonRepeat {
    public static void main(String[] args) {
        String s="swiss"; Map<Character,Integer> m=new LinkedHashMap<>();
        for (char c:s.toCharArray()) m.put(c,m.getOrDefault(c,0)+1);
        m.forEach((k,v)->{ if(v==1){System.out.println(k); return;} });
    }
}
```

■ Output / Example:

Input: swiss → Output: w

■ 39. Max Occurring Character

```
import java.util.HashMap;
public class MaxChar {
    public static void main(String[] args) {
        String s="programming"; HashMap<Character,Integer> m=new HashMap<>();
        for (char c:s.toCharArray()) m.put(c,m.getOrDefault(c,0)+1);
        char mx=s.charAt(0); int mc=0;
        for (char c:m.keySet()) if(m.get(c)>mc){mc=m.get(c);mx=c;}
        System.out.println("Max: "+mx+" (" +mc+" times)");
    }
}
```

■ Output / Example:

Input: programming → Output: Max: g (2 times)

■ 40. Double Each Character

```
public class DoubleChars {
    public static void main(String[] args) {
        StringBuilder sb=new StringBuilder();
        for (char c:"hello".toCharArray()) sb.append(c).append(c);
        System.out.println(sb);
    }
}
```

■ Output / Example:

Input: hello → Output: hheelllloo

■ 41. Swap Two Strings

```
public class SwapStrings {
    public static void main(String[] args) {
        String a="Hello", b="World";
        a=a+b; b=a.substring(0,a.length()-b.length()); a=a.substring(b.length());
        System.out.println("a="+a+", b="+b);
    }
}
```

■ Output / Example:

Output: a=World, b=Hello

■ 42. Print Even Indexed Characters

```
public class EvenIndex {
    public static void main(String[] args) {
        String s="Automation";
        for (int i=0;i<s.length();i++) if(i%2==0) System.out.print(s.charAt(i));
    }
}
```

■ Output / Example:

Output: Atmto

■ 43. String Length, isEmpty, contains

```
public class StringUtils {
    public static void main(String[] args) {
        String s="Automation Testing";
        System.out.println("Length: "+s.length());
        System.out.println("isEmpty: "+s.isEmpty());
        System.out.println("contains 'Testing': "+s.contains("Testing"));
    }
}
```

■ Output / Example:

Output: Length:18 isEmpty:false contains:true

■ 44. Count Char Occurrences

```
public class CountChar {
    public static void main(String[] args) {
        String s="Automation"; char ch='a'; int c=0;
        for (char x:s.toCharArray()) if(x==ch) c++;
        System.out.println("'" +ch+" found "+c+" time(s).");
    }
}
```

■ Output / Example:

Output: 'a' found 1 time(s).

■ 45. Reverse Words Keeping Digits

```
import java.util.regex.*;
public class ReverseWordsKeepDigits {
    public static void main(String[] args) {
        String s="test123abc"; StringBuilder r=new StringBuilder();
        Matcher m=Pattern.compile("[a-zA-Z]+|\\d+").matcher(s);
        while(m.find()){String p=m.group(); r.append(p.matches("[a-zA-Z]+")?new StringBuilder(p).reverse():p);}
        System.out.println(r);
    }
}
```

■ Output / Example:

Input: test123abc → Output: tset123cba

■ 46. Lambda – Filter Even Numbers

```
import java.util.*; import java.util.stream.*;
public class LambdaFilter {
    public static void main(String[] args) {
        List<Integer> nums=Arrays.asList(1,2,3,4,5,6,7,8,9,10);
        List<Integer> evens=nums.stream().filter(n->n%2==0).collect(Collectors.toList());
        System.out.println(evens);
    }
}
```

■ Output / Example:

Output: [2, 4, 6, 8, 10]

■ 47. Simple Login

```
import java.util.Scanner;
public class SimpleLogin {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        String u=sc.nextLine(), p=sc.nextLine();
        System.out.println("admin".equals(u)&&"pass123".equals(p)?"Login OK":"Login FAILED");
    }
}
```

■ Output / Example:

Correct creds → Login OK | Wrong creds → Login FAILED

■ 3 Arrays & Collections

■ 48. Sort Array – Built-in

```
import java.util.Arrays;
public class SortBuiltin {
    public static void main(String[] args) {
        int[] a={5,2,8,1,3}; Arrays.sort(a);
        System.out.println(Arrays.toString(a));
    }
}
```

■ Output / Example:

Output: [1, 2, 3, 5, 8]

■ 49. Selection Sort (No Built-in)

```
public class SelectionSort {
    public static void main(String[] args) {
        int[] a={5,2,9,1,6}; int n=a.length;
        for(int i=0;i<n-1;i++){int m=i;for(int j=i+1;j<n;j++) if(a[j]<a[m]) m=j;
        int t=a[i];a[i]=a[m];a[m]=t;}
        for(int x:a) System.out.print(x+" ");
    }
}
```

■ Output / Example:

Output: 1 2 5 6 9

■ 50. Bubble Sort

```
public class BubbleSort {
    public static void main(String[] args) {
        int[] a={64,34,25,12,22,11,90}; int n=a.length;
        for(int i=0;i<n-1;i++) for(int j=0;j<n-i-1;j++) if(a[j]>a[j+1]){int t=a[j];a[j]=a[j+1];a[j+1]=t;}
        for(int x:a) System.out.print(x+" ");
    }
}
```

■ Output / Example:

Output: 11 12 22 25 34 64 90

■ 51. Largest and Smallest

```
public class MinMax {
    public static void main(String[] args) {
        int[] a={5,2,9,1,6}; int mn=a[0],mx=a[0];
        for(int x:a){if(x<mn)mn=x;if(x>mx)mx=x;}
        System.out.println("Min:"+mn+" Max:"+mx);
    }
}
```

■ Output / Example:

Output: Min:1 Max:9

■ 52. Second Largest Element

```
public class SecondLargest {
    public static void main(String[] args) {
        int[] a={12,35,1,10,34};
        int f=Integer.MIN_VALUE, s=Integer.MIN_VALUE;
        for(int n:a){if(n>f){s=f;f=n;}else if(n>s&&f!=s)s=n;}
        System.out.println("2nd Largest: "+s);
    }
}
```

■ Output / Example:

Output: 2nd Largest: 34

■ 53. Remove Duplicates

```
import java.util.HashSet;
public class RemoveDups {
    public static void main(String[] args) {
        int[] a={1,2,2,3,4,4}; HashSet<Integer> s=new HashSet<>();
        for(int x:a) s.add(x);
        System.out.println("Unique: "+s);
    }
}
```

■ Output / Example:

Output: Unique: [1, 2, 3, 4]

■ 54. Find Common Elements

```
import java.util.HashSet;
public class CommonElems {
    public static void main(String[] args) {
        int[] a={1,2,3,4}, b={3,4,5,6}; HashSet<Integer> s=new HashSet<>();
        for(int x:a) s.add(x);
        for(int x:b) if(s.contains(x)) System.out.print(x+" ");
    }
}
```

■ Output / Example:

Output: 3 4

■ 55. Merge Two Arrays

```
import java.util.Arrays;
public class MergeArrays {
    public static void main(String[] args) {
        int[] a={1,3,5}, b={2,4,6}, m=new int[a.length+b.length];
        System.arraycopy(a,0,m,0,a.length); System.arraycopy(b,0,m,a.length,b.length);
        System.out.println(Arrays.toString(m));
    }
}
```

■ Output / Example:

Output: [1, 3, 5, 2, 4, 6]

■ 56. Linear Search

```
public class LinearSearch {
    public static void main(String[] args) {
        int[] a={5,2,9,1,6}; int tgt=6;
        for(int i=0;i<a.length;i++) if(a[i]==tgt){System.out.println("Found at "+i);return;}
        System.out.println("Not found");
    }
}
```

■ Output / Example:

Output: Found at 4

■ 57. Sum Integers from Mixed Array

```
public class SumMixed {
    public static void main(String[] args) {
        String[] a={"5","2","a","9","#"}; int sum=0;
        for(String s:a){try{sum+=Integer.parseInt(s);}catch(NumberFormatException e){}}
        System.out.println("Sum: "+sum);
    }
}
```

■ Output / Example:

Output: Sum: 16

■ 58. Count Odd and Even

```
public class CountOddEven {
    public static void main(String[] args) {
        int[] a={1,2,3,4,5,6,7,8,9}; int odd=0,even=0;
        for(int x:a){if(x%2==0)even++;else odd++;}
        System.out.println("Even:"+even+" Odd:"+odd);
    }
}
```

■ Output / Example:

Output: Even:4 Odd:5

■ 59. Non-Repeated Elements

```
import java.util.*;
public class NonRepeated {
    public static void main(String[] args) {
        int[] a={1,1,2,2,3,4,5,5}; Map<Integer,Integer> m=new LinkedHashMap<>();
        for(int x:a) m.put(x,m.getOrDefault(x,0)+1);
        m.forEach((k,v)->{ if(v==1) System.out.print(k+" "); });
    }
}
```

■ Output / Example:

Input: {1,1,2,2,3,4,5,5} → Output: 3 4

■ 60. First and Last of ArrayList

```
import java.util.*;
public class FirstLast {
    public static void main(String[] args) {
        List<String> l=Arrays.asList("Apple","Banana","Cherry");
        System.out.println("First:"+l.get(0)+" Last:"+l.get(l.size()-1));
    }
}
```

■ Output / Example:

Output: First:Apple Last:Cherry

■ 61. Find Minimum and Maximum

```
import java.util.Arrays;
public class FindMinMax {
    public static void main(String[] args) {
        int[] a={5,2,9,1,6};
        System.out.println("Min:"+Arrays.stream(a).min().getAsInt());
        System.out.println("Max:"+Arrays.stream(a).max().getAsInt());
    }
}
```

■ Output / Example:

Output: Min:1 Max:9

■ 62. Comparable vs Comparator

```
import java.util.*;
class Employee implements Comparable<Employee> {
    String name; int salary;
    Employee(String n,int s){name=n;salary=s;}
    public int compareTo(Employee e){return this.salary-e.salary;} // natural order
    public String toString(){return name+"("+salary+")";}
}
public class CompareDemo {
    public static void main(String[] args) {
        List<Employee> list=Arrays.asList(
            new Employee("Bob",50000),new Employee("Alice",70000),new Employee("Charlie",40000));
        Collections.sort(list); // uses Comparable
        System.out.println("By salary: "+list);
        list.sort(Comparator.comparing(e->e.name)); // uses Comparator
        System.out.println("By name: "+list);
    }
}
```

■ Output / Example:

By salary: [Charlie(40000), Bob(50000), Alice(70000)]
By name: [Alice(70000), Bob(50000), Charlie(40000)]

■ 4. OOP Core Concepts

■ 63. Encapsulation

```
// Encapsulation: hide data, expose via getters/setters
public class BankAccount {
    private double balance; // private field
    public BankAccount(double b) { this.balance = b; }
    public double getBalance() { return balance; } // getter
    public void deposit(double amt) { // setter logic
        if (amt > 0) balance += amt;
    }
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000);
        acc.deposit(500);
        System.out.println("Balance: " + acc.getBalance()); // 1500
    }
}
```

■ Output / Example:

Output: Balance: 1500.0

■ 64. Inheritance

```
class Vehicle {
    String brand; int speed;
    Vehicle(String b, int s) { brand=b; speed=s; }
    void display() { System.out.println(brand+" goes "+speed+" km/h"); }
}
class Car extends Vehicle {
    int doors;
    Car(String b, int s, int d) { super(b,s); doors=d; }
    void display() { super.display(); System.out.println("Doors: "+doors); }
}
public class InheritanceDemo {
    public static void main(String[] args) {
        Car c = new Car("Toyota", 180, 4);
        c.display();
    }
}
```

■ Output / Example:

```
Output: Toyota goes 180 km/h
Doors: 4
```

■ 65. Polymorphism – Method Overriding & Overloading

```
class Shape {
    double area() { return 0; } // overridden by subclasses
}
class Circle extends Shape {
    double r; Circle(double r){this.r=r;}
    double area() { return Math.PI*r*r; }
}
class Rectangle extends Shape {
    double w,h; Rectangle(double w,double h){this.w=w;this.h=h;}
    double area() { return w*h; }
}
// Overloading: same method name, different params
class MathUtils {
    static int add(int a,int b){return a+b;}
    static double add(double a,double b){return a+b;}
    static int add(int a,int b,int c){return a+b+c;}
}
public class PolyDemo {
    public static void main(String[] args) {
        Shape[] shapes={new Circle(5), new Rectangle(4,6)};
        for(Shape s:shapes) System.out.printf("Area: %.2f\n",s.area());
        System.out.println(MathUtils.add(2,3)); // 5
        System.out.println(MathUtils.add(2.5,3.5)); // 6.0
        System.out.println(MathUtils.add(1,2,3)); // 6
    }
}
```

■ Output / Example:

```
Area: 78.54
Area: 24.00
5 | 6.0 | 6
```


■ 66. Abstraction – Abstract Class vs Interface

```
// Abstract class: partial implementation, can have state
abstract class Animal {
    String name;
    Animal(String n){name=n;}
    abstract void sound(); // must override
    void breathe(){System.out.println(name+" breathes.");}
}

// Interface: pure contract, all methods public abstract by default
interface Swimmable { void swim(); }
interface Flyable { void fly(); }

class Duck extends Animal implements Swimmable, Flyable {
    Duck(String n){super(n);}
    public void sound(){System.out.println(name+": Quack!);}
    public void swim() {System.out.println(name+" swims.");}
    public void fly() {System.out.println(name+" flies.");}
}

public class AbstractDemo {
    public static void main(String[] args) {
        Duck d=new Duck("Donald");
        d.sound(); d.breathe(); d.swim(); d.fly();
    }
}
```

■ Output / Example:

```
Donald: Quack!
Donald breathes.
Donald swims.
Donald flies.
```

■ 67. Immutable Class

```
// Immutable: once created, state cannot change
// Rules: final class, private final fields, no setters, deep copy mutable fields
import java.util.Collections;
import java.util.List;
import java.util.ArrayList;
public final class ImmutableEmployee {
    private final int id;
    private final String name;
    private final List<String> skills; // mutable field – needs deep copy

    public ImmutableEmployee(int id, String name, List<String> skills) {
        this.id = id; this.name = name;
        this.skills = Collections.unmodifiableList(new ArrayList<>(skills)); // defensive copy
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public List<String> getSkills() { return skills; }

    public static void main(String[] args) {
        List<String> s=new ArrayList<>(); s.add("Java"); s.add("Selenium");
        ImmutableEmployee e=new ImmutableEmployee(1,"Alice",s);
        s.add("Hacking"); // does NOT affect e.skills
        System.out.println(e.getSkills()); // [Java, Selenium]
    }
}
```

■ Output / Example:

```
Output: [Java, Selenium]
```

■ 68. String Pool & StringBuilder vs StringBuffer

```
public class StringPool {
    public static void main(String[] args) {
        // String Pool – literals share memory
        String a="hello", b="hello";
        System.out.println(a==b); // true – same pool ref
        String c=new String("hello");
        System.out.println(a==c); // false – heap object
        System.out.println(a.equals(c)); // true – same content

        // StringBuilder (NOT thread-safe, faster)
        StringBuilder sb=new StringBuilder("Hello");
        sb.append(" World"); sb.insert(5,","); sb.delete(0,6);
        System.out.println(sb); // World

        // StringBuffer (thread-safe, slower)
        StringBuffer sf=new StringBuffer("Hello");
        sf.reverse();
        System.out.println(sf); // olleH
    }
}
```

■ Output / Example:

```
true false true
World
olleH
```

■ 69. Generics

```
// Generic class – works with any type
class Pair<K, V> {
    private K key; private V value;
    Pair(K k, V v){key=k; value=v;}
    public K getKey(){return key;}
    public V getValue(){return value;}
    public String toString(){return key+" -> "+value;}
}

// Generic method
class GenUtils {
    static <T extends Comparable<T>> T max(T a, T b){return a.compareTo(b)>0?a:b;}
}

public class GenericsDemo {
    public static void main(String[] args) {
        Pair<String,Integer> p=new Pair<>("Alice",95);
        System.out.println(p); // Alice -> 95
        System.out.println(GenUtils.max(10,20)); // 20
        System.out.println(GenUtils.max("A","Z")); // Z
    }
}
```

■ Output / Example:

```
Alice -> 95
20
Z
```

■ 5. Design Patterns

■ 70. Singleton Pattern

```
// Ensures only ONE instance exists – used for WebDriver, DB connections
public class WebDriverManager {
    private static WebDriverManager instance; // single instance
    private String browser;

    private WebDriverManager() { browser="Chrome"; } // private constructor

    // Thread-safe lazy initialization (double-checked locking)
    public static WebDriverManager getInstance() {
        if (instance == null) {
            synchronized (WebDriverManager.class) {
                if (instance == null) instance = new WebDriverManager();
            }
        }
        return instance;
    }

    public String getBrowser(){ return browser; }
    public void setBrowser(String b){ browser=b; }

    public static void main(String[] args) {
        WebDriverManager d1=WebDriverManager.getInstance();
        WebDriverManager d2=WebDriverManager.getInstance();
        System.out.println(d1==d2); // true – same instance
        d1.setBrowser("Firefox");
        System.out.println(d2.getBrowser()); // Firefox
    }
}
```

■ Output / Example:

```
true
Firefox
```

■ 71. Factory Pattern

```
// Creates objects without exposing creation logic
interface Browser { void open(String url); }

class Chrome implements Browser { public void open(String u){System.out.println("Chrome: "+u);} }
class Firefox implements Browser { public void open(String u){System.out.println("Firefox: "+u);} }
class Safari implements Browser { public void open(String u){System.out.println("Safari: "+u);} }

class BrowserFactory {
    public static Browser getBrowser(String type) {
        switch (type.toLowerCase()) {
            case "chrome": return new Chrome();
            case "firefox": return new Firefox();
            case "safari": return new Safari();
            default: throw new IllegalArgumentException("Unknown browser: "+type);
        }
    }
}

public class FactoryDemo {
    public static void main(String[] args) {
        Browser b=BrowserFactory.getBrowser("firefox");
        b.open("https://www.example.com");
    }
}
```

■ Output / Example:

```
Output: Firefox: https://www.example.com
```

■ 72. Builder Pattern

```
// Constructs complex objects step by step – used for test data, API requests
public class TestUser {
    private final String name, email, role;
    private final int age;

    private TestUser(Builder b){name=b.name;email=b.email;role=b.role;age=b.age;}

    public static class Builder {
        private String name,email,role="viewer"; private int age=0;
        public Builder name(String n){name=n;return this;}
        public Builder email(String e){email=e;return this;}
        public Builder role(String r){role=r;return this;}
        public Builder age(int a){age=a;return this;}
        public TestUser build(){return new TestUser(this);}
    }

    public String toString(){return name+" | "+email+" | "+role+" | age:"+age;}

    public static void main(String[] args) {
        TestUser u=new TestUser.Builder()
            .name("Alice").email("alice@test.com").role("admin").age(30)
            .build();
        System.out.println(u);
    }
}
```

■ Output / Example:

Output: Alice | alice@test.com | admin | age:30

■ 73. Page Object Model (POM) Pattern

```
import org.openqa.selenium.*; import org.openqa.selenium.support.*;

// Page class encapsulates all element locators & actions for a page
public class LoginPage {
    private WebDriver driver;

    @FindBy(id="username") private WebElement usernameField;
    @FindBy(id="password") private WebElement passwordField;
    @FindBy(id="loginBtn") private WebElement loginButton;
    @FindBy(id="errorMsg") private WebElement errorMessage;

    public LoginPage(WebDriver driver) {
        this.driver=driver;
        PageFactory.initElements(driver,this); // initialise @FindBy elements
    }

    public void login(String user,String pass) {
        usernameField.sendKeys(user);
        passwordField.sendKeys(pass);
        loginButton.click();
    }

    public String getErrorMessage() { return errorMessage.getText(); }
}

// Usage in Test class:
// LoginPage lp = new LoginPage(driver);
// lp.login("admin","pass123");
// Assert.assertEquals(lp.getErrorMessage(), "Invalid credentials");
```

■ Output / Example:

Concept: encapsulates locators & actions – keeps tests clean

■ 74. Strategy Pattern

```
// Swap algorithms at runtime – e.g., different sorting/reporting strategies
interface SortStrategy { void sort(int[] data); }

class BubbleSortStrategy implements SortStrategy {
    public void sort(int[] d){
        for(int i=0;i<d.length-1;i++) for(int j=0;j<d.length-i-1;j++)
            if(d[j]>d[j+1]){int t=d[j];d[j]=d[j+1];d[j+1]=t;}
        System.out.println("Bubble sorted");
    }
}

class QuickSortStrategy implements SortStrategy {
    public void sort(int[] d){ java.util.Arrays.sort(d); System.out.println("Quick sorted"); }
}

class Sorter {
    private SortStrategy strategy;
    Sorter(SortStrategy s){strategy=s;}
    void setStrategy(SortStrategy s){strategy=s;}
    void sort(int[] d){strategy.sort(d);}
}

public class StrategyDemo {
    public static void main(String[] args) {
        int[] data={5,3,1,4,2};
        Sorter sorter=new Sorter(new BubbleSortStrategy());
        sorter.sort(data);
        sorter.setStrategy(new QuickSortStrategy()); // switch strategy at runtime
        sorter.sort(data);
    }
}
```

■ *Output / Example:*

```
Bubble sorted
Quick sorted
```

■ 75. Observer Pattern

```
// One-to-many dependency – used in event listeners, test reporting
import java.util.*;

interface TestObserver { void onEvent(String event); }

class ExtentReporter implements TestObserver {
    public void onEvent(String e){ System.out.println("[Extent] "+e); }
}

class SlackNotifier implements TestObserver {
    public void onEvent(String e){ System.out.println("[Slack] Notify: "+e); }
}

class TestRunner {
    private List<TestObserver> observers=new ArrayList<>();
    void addObserver(TestObserver o){observers.add(o);}
    void runTest(String testName){
        System.out.println("Running: "+testName);
        notify("PASS: "+testName);
    }
    private void notify(String event){ observers.forEach(o->o.onEvent(event)); }
}

public class ObserverDemo {
    public static void main(String[] args) {
        TestRunner tr=new TestRunner();
        tr.addObserver(new ExtentReporter());
        tr.addObserver(new SlackNotifier());
        tr.runTest("LoginTest");
    }
}
```

■ Output / Example:

```
Running: LoginTest
[Extent] PASS: LoginTest
[Slack] Notify: PASS: LoginTest
```

■ 6 SOLID & Exception Handling

■ 76. SOLID – Single Responsibility Principle

```
// BAD: one class doing too much
class BadReport { void fetchData(){} void format(){} void print(){} void email(){} }

// GOOD: each class has ONE job
class DataFetcher { void fetchData(){ System.out.println("Fetching data..."); } }
class ReportFormatter{ void format() { System.out.println("Formatting..."); } }
class ReportPrinter { void print() { System.out.println("Printing report..."); } }
class EmailSender { void send() { System.out.println("Emailing report..."); } }

public class SRPDemo {
    public static void main(String[] args) {
        new DataFetcher().fetchData();
        new ReportFormatter().format();
        new ReportPrinter().print();
        new EmailSender().send();
    }
}
```

■ Output / Example:

```
Fetching data...
Formatting...
Printing report...
Emailing report...
```

■ 77. SOLID – Open/Closed Principle

```
// Open for EXTENSION, Closed for MODIFICATION
interface Discount { double apply(double price); }

class NoDiscount implements Discount { public double apply(double p){return p;} }
class SeasonalDiscount implements Discount { public double apply(double p){return p*0.80;} }
class LoyaltyDiscount implements Discount { public double apply(double p){return p*0.90;} }
// Adding new discount = new class, not modifying existing ones

class PriceCalculator {
double calculate(double price, Discount d){ return d.apply(price); }
}

public class OCPDemo {
public static void main(String[] args) {
PriceCalculator pc=new PriceCalculator();
System.out.println(pc.calculate(100, new NoDiscount())); // 100.0
System.out.println(pc.calculate(100, new SeasonalDiscount())); // 80.0
System.out.println(pc.calculate(100, new LoyaltyDiscount())); // 90.0
}
}
```

■ Output / Example:

```
100.0
80.0
90.0
```

■ 78. SOLID – Interface Segregation Principle

```
// BAD: fat interface forces classes to implement unused methods
interface BadWorker { void work(); void eat(); void sleep(); }

// GOOD: split into focused interfaces
interface Workable { void work(); }
interface Eatable { void eat(); }
interface Sleepable { void sleep(); }

class Human implements Workable, Eatable, Sleepable {
public void work() { System.out.println("Human works"); }
public void eat() { System.out.println("Human eats"); }
public void sleep(){ System.out.println("Human sleeps"); }
}

class Robot implements Workable { // robots don't eat or sleep
public void work(){ System.out.println("Robot works"); }
}

public class ISPDemo {
public static void main(String[] args) {
new Human().work(); new Human().eat();
new Robot().work();
}
}
```

■ Output / Example:

```
Human works
Human eats
Robot works
```

■ 79. Exception Handling – Custom Exception & Hierarchy

```
// Custom checked exception
class InvalidAgeException extends Exception {
InvalidAgeException(String msg){ super(msg); }
}

// Custom unchecked exception
class InsufficientFundsException extends RuntimeException {
double amount;
InsufficientFundsException(double a){ super("Short by: "+a); amount=a; }
}

public class ExceptionDemo {
static void validateAge(int age) throws InvalidAgeException {
if (age<18) throw new InvalidAgeException("Age "+age+" is below 18");
}

static void withdraw(double bal, double amt) {
if (amt>bal) throw new InsufficientFundsException(amt-bal);
System.out.println("Withdrawn: "+amt);
}

public static void main(String[] args) {
// Checked exception – must handle
try { validateAge(15); }
catch (InvalidAgeException e){ System.out.println("Caught: "+e.getMessage()); }
// Unchecked exception
try { withdraw(100, 200); }
catch (InsufficientFundsException e){ System.out.println("Caught: "+e.getMessage()); }
// finally always runs
try { int x=1/0; }
catch (ArithmeticException e){ System.out.println("Division error"); }
finally { System.out.println("Finally block runs"); }
}
}
```

■ Output / Example:

```
Caught: Age 15 is below 18
Caught: Short by: 100.0
Division error
Finally block runs
```

■ 80. try-with-resources (AutoCloseable)

```
// Automatically closes resource – no need for finally block
class DBConnection implements AutoCloseable {
DBConnection(){ System.out.println("Connection opened"); }
void query(String sql){ System.out.println("Executing: "+sql); }
public void close(){ System.out.println("Connection closed"); }
}

public class TryWithResources {
public static void main(String[] args) {
try (DBConnection db = new DBConnection()) {
db.query("SELECT * FROM users");
// db.close() called automatically, even if exception occurs
}
System.out.println("After try block");
}
}
```

■ Output / Example:

```
Connection opened
Executing: SELECT * FROM users
Connection closed
After try block
```

■ 7. Multithreading, Java 8 & Advanced

■ 81. Multithreading – Thread & Runnable

```
public class ThreadDemo {
// Way 1: extend Thread
static class MyThread extends Thread {
public void run(){ System.out.println("Thread: "+getName()); }
}
// Way 2: implement Runnable (preferred – allows extending other class)
static class MyRunnable implements Runnable {
public void run(){ System.out.println("Runnable: "+Thread.currentThread().getName()); }
}
public static void main(String[] args) throws InterruptedException {
MyThread t1=new MyThread(); t1.start();
Thread t2=new Thread(new MyRunnable()); t2.start();
// Way 3: Lambda (Java 8)
Thread t3=new Thread()->System.out.println("Lambda thread"); t3.start();
t1.join(); t2.join(); t3.join(); // wait for all
}
}
```

■ Output / Example:

```
Thread: Thread-0
Runnable: Thread-1
Lambda thread
```

■ 82. Synchronized – Prevent Race Condition

```
public class Counter {
private int count=0;
// synchronized ensures only one thread can execute at a time
public synchronized void increment(){ count++; }
public int getCount(){ return count; }

public static void main(String[] args) throws InterruptedException {
Counter c=new Counter();
Thread t1=new Thread()->{ for(int i=0;i<1000;i++) c.increment(); };
Thread t2=new Thread()->{ for(int i=0;i<1000;i++) c.increment(); };
t1.start(); t2.start(); t1.join(); t2.join();
System.out.println("Count: "+c.getCount()); // always 2000 with sync
}
}
```

■ Output / Example:

```
Output: Count: 2000 (without synchronized, result is unpredictable)
```

■ 83. Collections – HashMap Internal & ConcurrentHashMap

```
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
public class CollectionsDemo {
    public static void main(String[] args) {
        // HashMap – not thread-safe, allows one null key
        HashMap<String,Integer> hm=new HashMap<>();
        hm.put("A",1); hm.put("B",2); hm.put(null,0);
        System.out.println("HashMap: "+hm);

        // LinkedHashMap – maintains insertion order
        LinkedHashMap<String,Integer> lhm=new LinkedHashMap<>();
        lhm.put("B",2); lhm.put("A",1); lhm.put("C",3);
        System.out.println("LinkedHashMap: "+lhm); // B, A, C

        // TreeMap – sorted by key
        TreeMap<String,Integer> tm=new TreeMap<>(lhm);
        System.out.println("TreeMap: "+tm); // A, B, C

        // ConcurrentHashMap – thread-safe, no null keys/values
        ConcurrentHashMap<String,Integer> chm=new ConcurrentHashMap<>();
        chm.put("X",10); chm.put("Y",20);
        System.out.println("ConcurrentHashMap: "+chm);
    }
}
```

■ Output / Example:

```
HashMap: {null=0, A=1, B=2}
LinkedHashMap: {B=2, A=1, C=3}
TreeMap: {A=1, B=2, C=3}
ConcurrentHashMap: {X=10, Y=20}
```

■ 84. Collections – List vs Set vs Queue

```
import java.util.*;
public class CollectionTypes {
    public static void main(String[] args) {
        // ArrayList – ordered, allows duplicates, O(1) get
        List<String> al=new ArrayList<>(Arrays.asList("B","A","C","A"));
        System.out.println("ArrayList: "+al); // [B, A, C, A]

        // HashSet – no duplicates, no order, O(1) add/contains
        Set<String> hs=new HashSet<>(al);
        System.out.println("HashSet: "+hs); // unordered, no dup

        // LinkedHashSet – no duplicates, maintains insertion order
        Set<String> lhs=new LinkedHashSet<>(al);
        System.out.println("LinkedHashSet: "+lhs); // [B, A, C]

        // TreeSet – no duplicates, sorted
        Set<String> ts=new TreeSet<>(al);
        System.out.println("TreeSet: "+ts); // [A, B, C]

        // PriorityQueue – retrieves in natural/custom order
        Queue<Integer> pq=new PriorityQueue<>(Arrays.asList(5,1,3,2,4));
        System.out.println("PQ poll: "+pq.poll()); // 1 (smallest first)
    }
}
```

■ Output / Example:

```
ArrayList: [B, A, C, A]
LinkedHashSet: [B, A, C]
TreeSet: [A, B, C]
PQ poll: 1
```

■ 85. Java 8 – Streams API

```
import java.util.*;
import java.util.stream.*;
public class StreamsDemo {
    public static void main(String[] args) {
        List<Integer> nums=Arrays.asList(1,2,3,4,5,6,7,8,9,10);

        // filter + map + collect
        List<Integer> result=nums.stream()
            .filter(n->n%2==0) // keep evens
            .map(n->n*n) // square them
            .collect(Collectors.toList());
        System.out.println("Even squares: "+result);

        // reduce – sum all
        int sum=nums.stream().reduce(0,Integer::sum);
        System.out.println("Sum: "+sum);

        // sorted + distinct + limit
        List<Integer> data=Arrays.asList(3,1,4,1,5,9,2,6,5);
        List<Integer> top3=data.stream().distinct().sorted().limit(3)
            .collect(Collectors.toList());
        System.out.println("Top 3 distinct sorted: "+top3);

        // groupingBy
        Map<Boolean,List<Integer>> partitioned=nums.stream()
            .collect(Collectors.partitioningBy(n->n%2==0));
        System.out.println("Even: "+partitioned.get(true));
        System.out.println("Odd: "+partitioned.get(false));
    }
}
```

■ Output / Example:

```
Even squares: [4, 16, 36, 64, 100]
Sum: 55
Top 3 distinct sorted: [1, 2, 3]
Even: [2,4,6,8,10] Odd: [1,3,5,7,9]
```

■ 86. Java 8 – Optional

```
import java.util.Optional;
public class OptionalDemo {
    static Optional<String> findUser(int id) {
        if (id==1) return Optional.of("Alice");
        return Optional.empty(); // no NullPointerException risk
    }
    public static void main(String[] args) {
        // Safe retrieval
        Optional<String> user=findUser(1);
        System.out.println(user.isPresent()); // true
        System.out.println(user.get()); // Alice
        System.out.println(user.orElse("Unknown")); // Alice

        Optional<String> missing=findUser(99);
        System.out.println(missing.orElse("Unknown")); // Unknown
        System.out.println(missing.orElseGet(()->"Guest")); // Guest

        // map & filter on Optional
        user.map(String::toUpperCase).ifPresent(System.out::println); // ALICE
    }
}
```

■ Output / Example:

```
true
Alice
Alice
Unknown
Guest
ALICE
```

■ 87. Java 8 – Functional Interfaces

```
import java.util.function.*;
public class FunctionalDemo {
    public static void main(String[] args) {
        // Predicate<T>: T -> boolean
        Predicate<Integer> isEven = n -> n%2==0;
        System.out.println(isEven.test(4)); // true
        System.out.println(isEven.negate().test(4)); // false

        // Function<T,R>: T -> R
        Function<String,Integer> strLen = String::length;
        System.out.println(strLen.apply("Automation")); // 10

        // BiFunction<T,U,R>: (T,U) -> R
        BiFunction<Integer,Integer,Integer> mul = (a,b) -> a*b;
        System.out.println(mul.apply(3,4)); // 12

        // Consumer<T>: T -> void
        Consumer<String> print = System.out::println;
        print.accept("Hello SDET!");

        // Supplier<T>: () -> T
        Supplier<Double> random = Math::random;
        System.out.println(random.get() > 0); // true
    }
}
```

■ Output / Example:

```
true
false
10
12
Hello SDET!
true
```

■ 88. Java 8 – Default & Static Interface Methods

```
interface Vehicle {
    void accelerate(); // abstract – must implement
    default void horn() { // default – optional override
        System.out.println("Beep beep!");
    }
    static Vehicle of(String type) { // static factory method in interface
        if("car".equals(type)) return ()->System.out.println("Car accelerates");
        return ()->System.out.println("Bike accelerates");
    }
}

class Car implements Vehicle {
    public void accelerate(){ System.out.println("Car: vroom!"); }
    // inherits default horn()
}

class ElectricCar implements Vehicle {
    public void accelerate(){ System.out.println("Electric: silent zoom!"); }
    public void horn(){ System.out.println("Silent horn!"); } // override default
}

public class DefaultMethodDemo {
    public static void main(String[] args) {
        Car c=new Car(); c.accelerate(); c.horn();
        ElectricCar e=new ElectricCar(); e.accelerate(); e.horn();
        Vehicle.of("bike").accelerate();
    }
}
```

■ Output / Example:

```
Car: vroom!
Beep beep!
Electric: silent zoom!
Silent horn!
Bike accelerates
```

■ 89. Multithreading – ExecutorService & Future

```
import java.util.concurrent.*;

public class ExecutorDemo {
    public static void main(String[] args) throws Exception {
        // Thread pool – reuse threads, avoid cost of creating new ones
        ExecutorService pool = Executors.newFixedThreadPool(3);

        // Submit Runnable (no return value)
        pool.submit(()->System.out.println("Task 1: "+Thread.currentThread().getName()));

        // Submit Callable (returns a value)
        Future<Integer> future = pool.submit(()->{
            Thread.sleep(100);
            return 42;
        });
        System.out.println("Future result: "+future.get()); // blocks until done

        // Submit multiple tasks
        for(int i=2;i<=4;i++){
            final int id=i;
            pool.submit(()->System.out.println("Task "+id));
        }
        pool.shutdown(); // graceful shutdown
        pool.awaitTermination(5, TimeUnit.SECONDS);
    }
}
```

■ Output / Example:

```
Task 1: pool-1-thread-1
Future result: 42
Task 2, Task 3, Task 4 (any order)
```

■ 90. HashMap – Internal Working & hashCode>equals

```
import java.util.Objects;
// Demonstrates WHY hashCode + equals contract matters in HashMap
class Point {
    int x, y;
    Point(int x,int y){this.x=x;this.y=y;}

    @Override public int hashCode(){ return Objects.hash(x,y); }
    // Without hashCode override, HashMap uses memory address –
    // two Point(1,2) would map to DIFFERENT buckets

    @Override public boolean equals(Object o){
        if(this==o) return true;
        if(!(o instanceof Point)) return false;
        Point p=(Point)o; return x==p.x&&y==p.y;
    }
}

public class HashMapInternal {
    public static void main(String[] args) {
        java.util.Map<Point,String> map=new java.util.HashMap<>();
        map.put(new Point(1,2),"Origin Area");
        // With proper hashCode+equals:
        System.out.println(map.get(new Point(1,2))); // Origin Area
        // Without them: null (different bucket)

        // HashMap capacity & load factor
        // Default capacity=16, load factor=0.75
        // When size > 16*0.75=12, it resizes (doubles to 32)
        System.out.println("HashMap key rules: non-null keys, one null allowed");
    }
}
```

■ Output / Example:

```
Origin Area
HashMap key rules: non-null keys, one null allowed
```